ARCHITECTURAL SPECIFICATION

RISE-Phish: Software Design Document (SDD)

IEEE Std 1016-2009 Compliant System Design for Multi-Representation Discrepancy-Aware Phishing Detection

--- SOFTWARE DESIGN DOCUMENT (SDD) ---

Submitted By:	[Student Name 1] (Roll No: [Roll No 1]) [Student Name 2] (Roll No: [Roll No 2]) [Student Name 3] (Roll No: [Roll No 3])
Project Guide / Supervisor:	[Project Guide Name] [Guide Designation & Department]
Department:	Department of Artificial Intelligence & Machine Learning / Computer Science
Institution:	[Institution / College / University Name]
Academic Year:	[Academic Year: 202X - 202Y]

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) specifies the complete software architecture, module decomposition, data models, algorithmic formulations, interface contracts, and security architecture of the RISE-Phish platform. The specification follows IEEE Std 1016-2009 and bridges the functional requirements documented in the SRS with the concrete codebase implementation.

1.2 Scope

The scope covers the backend FastAPI microservice (Python 3.12), the multi-representation compilation subsystem, the Cross-Representation Consistency Evaluation (CRCE) discrepancy engine, the Small Language Model (SLM) reasoning module, the deterministic evidence fusion engine, the SQLite persistence layer, the React 19 frontend dashboard, and the Chrome Manifest V3 extension.

1.3 Design Goals & Principles

- **Modularity & High Cohesion:** Each analytical step (crawling, OCR, Unicode, CRCE, SLM, Fusion) is encapsulated in an independent module with zero cyclic imports.
- **Local Execution & Privacy:** The system operates strictly within local workstation boundaries (calibrated for NVIDIA RTX 3050 4GB), preventing credential exfiltration.
- **Fault Tolerance & Graceful Degradation:** All analytical modules execute within bounded timeouts (15s for crawler, 1s for OCR/SLM) with automatic fallbacks.
- **Dual-Audience Explainability:** System outputs include both plain-language advice for everyday victims and an auditable forensic breakdown for SOC analysts.

2. SYSTEM ARCHITECTURAL OVERVIEW

2.1 Architectural Style

RISE-Phish follows a Layered Pipeline & Ports-and-Adapters (Hexagonal) architecture. The core analytical pipeline is isolated from external I/O adapters (FastAPI REST endpoints, Playwright headless browsers, Tesseract OCR binaries, and SQLite databases). Analytical results flow unidirectionally from acquisition through compilation, discrepancy computation, reasoning, fusion, and presentation.

2.2 Module Decomposition

The RISE-Phish codebase is decomposed into nine core modules located in backend/app/:

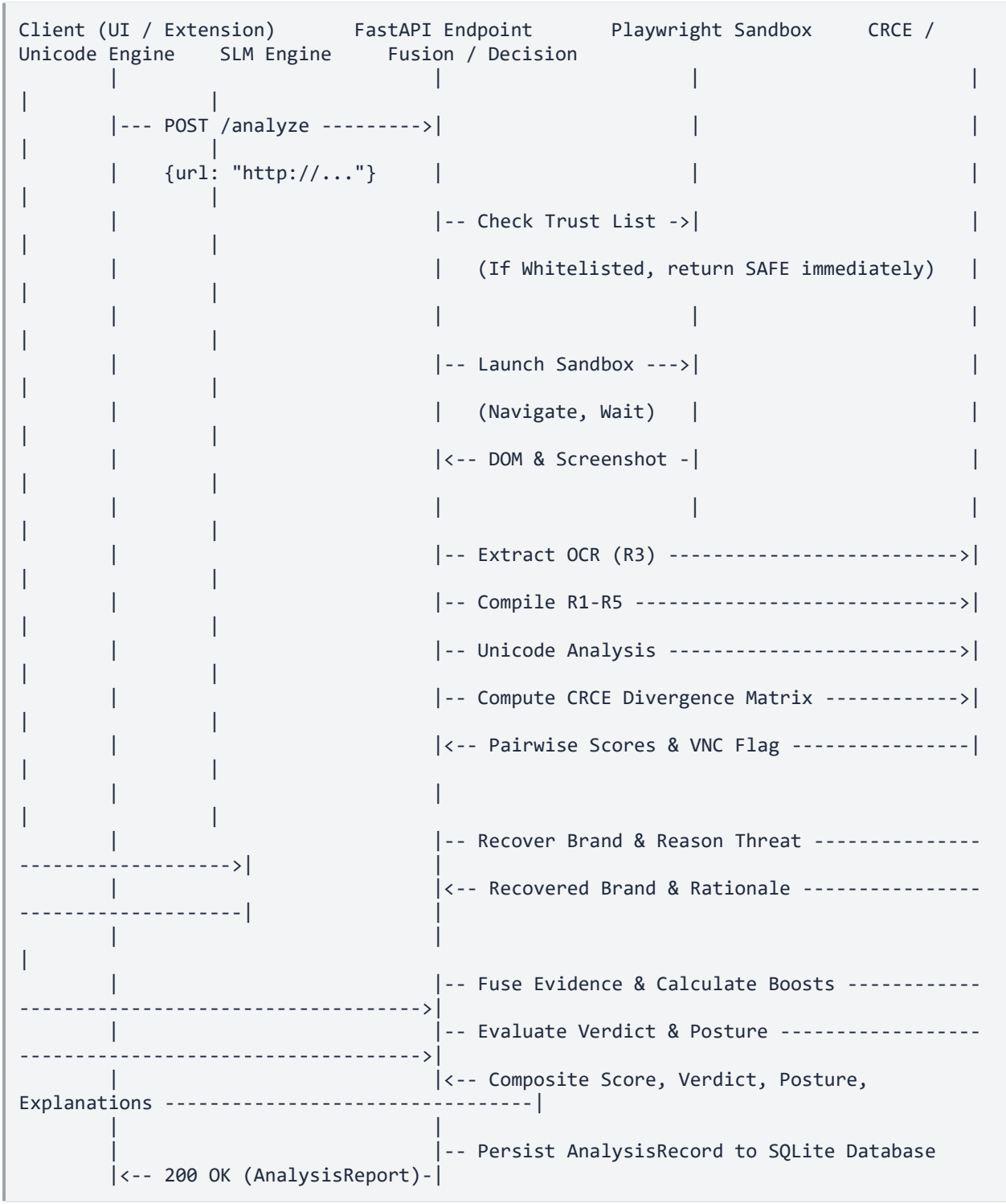
Module Name	Source File Path	Architectural Role & Key Responsibilities

Module Name	Source File Path	Architectural Role & Key Responsibilities
crawler.py	backend/app/core/crawler.py	Manages sandboxed Chromium instances via Playwright. Implements resource blocking (images/fonts when appropriate), timeout enforcement, viewport capture, and redirect tracing.
rep_builder.py	backend/app/core/rep_builder.py	Compiles raw DOM text (R1), canonical NFKC normalized text (R2), visual OCR text (R3), structural semantic intent (R4), and decomposed URL tokens (R5).
unicode_analyzer.py	backend/app/core/unicode_analyzer.py	Calculates character Shannon entropy, flags zero-width steganographic tokens, detects mixed-script codepoints, and maps Cyrillic/Greek confusables to Latin equivalents.
vision.py	backend/app/core/vision.py	Executes image pre-processing (grayscale contrast enhancement) and calls Tesseract OCR to extract visual ground truth from viewport screenshots.
crce.py	backend/app/core/crce.py	Computes the 5x5 pairwise discrepancy matrix using Normalized Edit Distance and Token Jaccard Divergence. Detects the Visual-Normalization Convergence (VNC) signature.
recovery.py	backend/app/slm/recovery.py	Resolves brand identity from normalized text and OCR against canonical enterprise dictionaries (Microsoft, PayPal, Google, Apple, Amazon, etc.).
reasoning.py	backend/app/slm/reasoning.py	Generates contextual threat justification using local SLM prompts or deterministic rule-based multi-factor evaluation.
fusion.py	backend/app/core/fusion.py	Fuses CRCE (35%), URL (25%), Unicode (25%), and SLM (15%) scores with deterministic non-linear interactive boosts.
decision.py	backend/app/core/decision.py	Maps composite scores to operational verdicts (SAFE, INVESTIGATE, PHISHING) and automated enforcement postures (ALLOW, WARN_USER, ISOLATE, BLOCK_ACCESS).
explainability.py	backend/app/core/explainability.py	Constructs dual structured outputs: a plain-language summary for end-users and an auditable forensic workbench for SOC analysts.

3. DATA FLOW AND SEQUENCE TRACES

3.1 Primary Execution Sequence: URL Analysis Workflow

Figure 3.1 illustrates the end-to-end execution sequence when a client requests an active URL scan:



Listing: Figure 3.1: Sequence Diagram of Primary URL Analysis Execution Pipeline

3.2 Secondary Execution Sequence: Extension Message & Email Analysis

When an end-user highlights an email snippet or webmail body in Chrome/Edge, the extension captures the selected text and submits it to POST /extension/analyze. The pipeline extracts embedded hyperlinks, executes Unicode confusable analysis on the raw text, identifies urgency and credential lure language, and computes a message-level risk score before the user clicks the link.

3.3 Error Recovery & Fallback Flow

- **Network Timeout / Unreachable Host:** If target domain fails DNS resolution or returns HTTP 4xx/5xx, crawler captures error state and forwards available URL tokens to syntactic analyzer.
- **OCR Processing Failure:** If Tesseract binary is missing or viewport image is blank, OCR text defaults to empty string; CRCE automatically reweights available DOM and URL pairs.
- **SLM Unavailability:** If SLM weights are uninitialized or GPU memory is saturated, the system seamlessly routes inference to the deterministic rule engine (reasoning.py), guaranteeing zero disruption.

4. DATA DESIGN AND PERSISTENT SCHEMAS

4.1 Database Architecture

Persistence is managed via SQLite (database.db) using SQLAlchemy 2.0 ORM. The relational model consists of two core tables:

Table Name	Column & Data Type	Semantic Description and Role
AnalysisRecord	id (VARCHAR(36) PK)	UUID primary key identifying the scan session.
AnalysisRecord	url (TEXT)	Target URL submitted for analysis.
AnalysisRecord	timestamp (DATETIME)	ISO 8601 UTC timestamp of scan execution.
AnalysisRecord	composite_threat_score (FLOAT)	Final fused threat score normalized to [0.0, 100.0].
AnalysisRecord	operational_verdict (VARCHAR(20))	Categorical decision: 'SAFE', 'INVESTIGATE', or 'PHISHING'.
AnalysisRecord	automated_posture (VARCHAR(30))	Enforcement posture: 'ALLOW', 'WARN_USER', 'ISOLATE', 'BLOCK_ACCESS'.
AnalysisRecord	representations_json (TEXT)	JSON blob storing extracted strings for R1, R2, R3, R4, and R5.
AnalysisRecord	discrepancies_json (TEXT)	JSON blob storing the 5x5 CRCE pairwise divergence distance matrix.

Table Name	Column & Data Type	Semantic Description and Role
AnalysisRecord	unicode_analysis_json (TEXT)	JSON blob containing confusable characters, entropy, and zero-width flags.
AnalysisRecord	slm_reasoning_json (TEXT)	JSON blob containing recovered brand, similarity score, and threat rationale.
AnalysisRecord	explanations_json (TEXT)	JSON blob containing dual-audience text (user advice and SOC breakdown).
FeedbackRecord	id (VARCHAR(36) PK)	UUID primary key for analyst feedback entry.
FeedbackRecord	analysis_id (VARCHAR(36) FK)	Foreign key referencing AnalysisRecord.id.
FeedbackRecord	user_label (VARCHAR(20))	Ground truth label assigned by analyst: 'SAFE' or 'PHISHING'.
FeedbackRecord	analyst_notes (TEXT)	Forensic justification or false positive reporting notes.
FeedbackRecord	created_at (DATETIME)	Timestamp of feedback submission.

4.2 Pydantic Data Contracts

All incoming REST payloads and analytical outputs are validated using Pydantic v2 models:

- **AnalysisRequest:** Validates incoming target URL, crawler timeout options, and optional manual raw text.
- **ExtensionScanRequest:** Validates target URL and extracted DOM snippet from the Chrome extension content script.
- **AnalysisReport:** Enforces complete JSON schema contract for API consumers, containing composite threat score, operational verdict, automated posture, representations dictionary, CRCE discrepancy matrix, unicode findings, and dual explanations.
- **FeedbackRequest:** Validates analyst ground-truth submissions, ensuring valid foreign keys and enum labels.

5. DETAILED ALGORITHMIC AND MATHEMATICAL DESIGN

5.1 Normalized Edit Distance & Token Jaccard Divergence

For any two representation strings A and B, the Cross-Representation Consistency Evaluation (CRCE) engine computes:

```
def compute_pairwise_divergence(text_a: str, text_b: str) -> float:
    if not text_a and not text_b:
```

```
        return 0.0
    if not text_a or not text_b:
        return 1.0

    # 1. Normalized Levenshtein Edit Distance (RapidFuzz C++)
    lev_dist = rapidfuzz.distance.Levenshtein.distance(text_a, text_b)
    ned = lev_dist / max(len(text_a), len(text_b))

    # 2. Token Jaccard Divergence
    tokens_a = set(text_a.lower().split())
    tokens_b = set(text_b.lower().split())
    if not tokens_a and not tokens_b:
        jaccard_div = 0.0
    else:
        intersection = len(tokens_a.intersection(tokens_b))
        union = len(tokens_a.union(tokens_b))
        jaccard_div = 1.0 - (intersection / union) if union > 0 else 0.0

    # Combined Symmetric Divergence Metric
    divergence = (0.5 * ned) + (0.5 * jaccard_div)
    return round(divergence, 4)
```

Listing: Listing 5.1: Pairwise Discrepancy Formulation in crce.py

5.2 Visual-Normalization Convergence (VNC) Detection Logic

The VNC signature is mathematically evaluated as:

```
def evaluate_vnc_signature(d_r1_r2: float, d_r1_r3: float, d_r2_r3: float) -> bool:
    # Condition: Raw DOM diverges from both Normalized and Visual representations,
    # while Normalized text converges with Visual perception.
    condition_dom_diverges = (d_r1_r2 >= 0.25) and (d_r1_r3 >= 0.25)
    condition_visual_converges = (d_r2_r3 <= 0.15)

    return bool(condition_dom_diverges and condition_visual_converges)
```

Listing: Listing 5.2: VNC Detection Algorithm in crce.py

6. SECURITY ARCHITECTURE

6.1 Playwright Crawler Sandboxing & SSRF Mitigation

Because the crawler accepts arbitrary user-supplied URLs, it introduces potential Server-Side Request Forgery (SSRF) risks. The security architecture enforces multi-layered defense-in-depth:

- **Browser Sandboxing:** Playwright executes with flags: --disable-gpu, --no-sandbox, --disable-setuid-sandbox, --disable-dev-shm-usage, --disable-extensions.
- **Loopback & Private Subnet Blocking:** Pre-navigation URL inspection validates that the destination IP does not resolve to private subnets (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, or 169.254.169.254).
- **Resource Limits & Download Termination:** Playwright instances enforce strict 15-second navigation caps and abort downloads of executables, archive files, or binary payloads.

6.2 Extension Manifest V3 Least-Privilege Architecture

The browser extension strictly complies with Chrome Web Store Manifest V3 policies:

- **Permission Minimization:** Requested permissions are minimized to: activeTab, storage, scripting, and contextMenus.
- **Content Security Policy (CSP):** No remote code execution; all scripts and styles are packaged locally within the extension archive.
- **Scoped Network Access:** Cross-origin communication is restricted to the local backend gateway at http://127.0.0.1:8000.